14 haftalık yol haritasında 5. hafta

ALGORİTMA VE PROGRAMLAMA – I · DERS NOTU

HAFTA 05

Karar Yapıları: Programın Yol Ayrımları

if, else-if, iç içe if, switch ve ?: ile karar veren programlar; De Morgan, girdi doğrulama, sınır değer testi ve kararların makine dilindeki karşılığı

MODÜL III · KONTROL AKIŞI · H5–H7

HAFTANIN BÜYÜK SORUSU

Tek bir fazladan satır ve eksik süslü parantezler, milyonlarca cihazın güvenli bağlantısını nasıl devre dışı bırakabilir?

Dr. Öğr. Üyesi Mehmet Sevri
Recep Tayyip Erdoğan Üniversitesi · Mühendislik ve Mimarlık Fakültesi · Bilgisayar Mühendisliği Bölümü
Bilgisayar Mühendisliği 1. sınıf · 2026–2027 Güz yarıyılı · Sürüm 1.0 · Eylül 2026
`Derleme ortamı: GCC (C17) · gcc -std=c17 -Wall -Wextra -pedantic`

İçindekiler

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları 2
1.1 Öğrenme çıktıları 3
2. Geçen Haftadan: Operatörlerden Kararlara 3
3. Kavramsal Anlatım 4
3.1 C'de doğruluk: 0 yanlış, sıfırdan farklı her değer doğru 4
3.2 if ve if-else; blok { } ve kapsam 4
3.3 else-if merdiveni ve koşulların sırası 5
3.4 İç içe if ve sallanan else 6
3.5 Bileşik koşullar: doğruluk tabloları, De Morgan ve kısa devre 7
3.6 Koşul operatörü ?: 8
3.7 switch-case: case, break, default ve bilinçli düşüş 8
3.8 Girdi doğrulama: savunmacı programlama 9
3.9 Test tasarımı: dal kapsamı ve sınır değer çözümlemesi 10
3.10 Kaputun Altında: kararlar makine dilinde 11
4. Çözümlü Örnekler: Örnek Merdiveni 12
5. Hata Avcısı 22
6. Mühendis Gözüyle 24
7. Mini-OBS Projesi: v0.5 24
8. Alıştırmalar ve Atölye 25
9. Kendini Değerlendirme 28
10. AP-2 Köprüsü ve Merak Kutusu 28
11. Özet, Kavram Haritası ve Terim Sözlüğü 29
12. Kaynaklar 30

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları

Tek bir fazladan satır ve eksik süslü parantezler, milyonlarca cihazın güvenli bağlantısını nasıl devre dışı bırakabilir?

Şimdiye kadar yazdığımız programlar hep aynı yolu izledi: satırlar yukarıdan aşağıya, sırayla çalıştı. Gerçek programlar ise sürekli karar verir: puan 60'ın üstünde mi, PIN doğru mu, bölen sıfır mı, imza geçerli mi? Bu hafta programa **yol ayrımları** ekleyen C yapılarını öğreneceğiz.

> GERÇEK DÜNYADAN • **Şubat 2014: "goto fail" (CVE-2014-1266)**
>
> Apple'ın SSL/TLS kitaplığı **SecureTransport**'ta, sunucunun anahtar değişimi iletisindeki imzayı doğrulayan işlevde (`SSLVerifySignedServerKeyExchange`) süslü parantezsiz bir `if`'in ardından `goto fail;` satırı **iki kez** yazılmıştı. `if` yalnızca ilk satırı koşula bağlıyordu; aynı girintideki ikinci satır **her durumda** çalışıyordu (Langley, 2014). Akış son imza doğrulama adımını hiç çalıştırmadan `fail` etiketine atlıyor, o anda hata

değişkeni 0 (başarı) olduğu için doğrulama başarılı sayılıyordu: aynı ağdaki bir saldırgan (ortadaki adam, man-in-the-middle) kendini güvenilir bir sunucu gibi gösterebiliyordu. Açık iOS 7.0.6, iOS 6.1.6, OS X 10.9.2 ve Apple TV 6.0.2 güncellemeleriyle kapatıldı (National Institute of Standards and Technology [NIST], 2014). Hatayı eğitsel bir yeniden kurguyla kendi makinemizde üreteceğiz (Bölüm 5, HA4); yanıtın özeti Bölüm 11.2'dedir.

1.1 Öğrenme çıktıları

ÖÇ 5.1 if, if-else, else-if merdiveni, iç içe if ve switch yapılarını problem türüne göre seçer ve uygular. *(Bölüm 3.2–3.4, 3.6–3.7, 4)*

ÖÇ 5.2 Bileşik koşulları mantıksal operatörler ve De Morgan kurallarıyla kurar ve sadeleştirir. *(Bölüm 3.1, 3.5)*

ÖÇ 5.3 Tüm dalları kapsayan test durumlarını sınır değer çözümlemesi ile tasarlar. *(Bölüm 3.8–3.9, 4.13)*

ÖÇ 5.4 Karar yapılarının makine düzeyindeki karşılığını (karşılaştırma, koşullu atlama, atlama tablosu) açıklar. *(Bölüm 3.10)*

Önkoşullar: H2'deki akış diyagramları ve karar sembolü (◇), H4'teki ilişkisel ve mantıksal operatörler. H2'de akış diyagramıyla çözdüğümüz **iki sayının büyüğü**, **üç sayının en büyüğü** ve **harf notu** problemlerini (H2 Örnek 3, 5, 6) bu hafta C'nin karar yapılarıyla derinleştiriyoruz.

2. Geçen Haftadan: Operatörlerden Kararlara

Karar yapılarının koşulu da bir ifadedir: `if (puan >= 60)` önce `puan >= 60` ifadesini hesaplar, sonra sonucun 0 olup olmadığına bakar. Üç soruyu önce kâğıt üzerinde yanıtlayın, sonra programı çalıştırın.

H05_K01_geri_cagirma.c · satır 10–22

```
10      /* S1: tamsayı bölmesi, kalan ve cast */
11      int a = 7;
12      int b = 2;
13      printf("%d %d %.1f\n", a / b, a % b, (double)a / b);
14
15      /* S2: ilişkisel ve mantıksal ifadeler 0 ya da 1 üretir */
16      int x = 5;
17      printf("%d %d %d\n", x > 3, x > 3 && x > 8, !x);
18
19      /* S3: artık yıl ifadesi (H4 Örnek 8) */
20      int yil = 1900;
21      int artik = (yil % 4 == 0 && yil % 100 != 0) || yil % 400 == 0;
22      printf("%d\n", artik);
```

TAHMİN ET · **Geri çağırma: üç soru**

- **S1.** İlk `printf` ne yazdırır? (tamsayı bölmesi, kalan ve cast)
- **S2.** `x = 5` iken `x > 3`, `x > 3 && x > 8` ve `!x` ifadelerinin değerleri nedir?
- **S3.** `yil = 1900` için artık yıl ifadesi hangi değeri üretir? Neden?

Yanıtlar (gerçek çıktı: `3 1 3.5` · `1 0 0` · `0`). S1: `7 / 2` tamsayı bölmesidir (3), `7 % 2` = 1, `(double)7 / 2` = 3.5. S2: ilişkisel ve mantıksal operatörler **int 0 ya da 1** üretir: 1, 0, 0 (`!5` = 0). S3: 1900 4'e ve 100'e bölünür ama 400'e bölünmez; `(1 && 0) || 0` = 0. Bu hafta aynı kuralı hem tek ifade hem de karar zinciri olarak

> yazacağız (Örnek 5).

3. Kavramsal Anlatım

3.1 C'de doğruluk: 0 yanlış, sıfırdan farklı her değer doğru

C'de **0 yanlış, sıfırdan farklı her değer doğrudur** (−7 de, 3 de doğrudur); karşılaştırma ve mantıksal operatörler sonucu `int` **0 ya da 1** olarak üretir. Bu yüzden `if (sayac)` ile `if (sayac != 0)` aynı anlama gelir. C99'un `<stdbool.h>` başlığı `bool` tipini ve `true` (1) / `false` (0) adlarını tanımlar; `bool` bir değişkenin yalnızca doğruluk değeri taşıdığını açıkça söyler.

H05_K02_dogruluk_degeri.c · satır 26–32

```
26      int sonuc = (10 > 3);
27      printf("(10 > 3) = %d, (10 < 3) = %d\n", sonuc, 10 < 3);
28
29      bool gecti = true;
30      bool devamsiz = false;
31      printf("gecti = %d, devamsiz = %d, sizeof(bool) = %zu\n",
32             gecti, devamsiz, sizeof(bool));
```

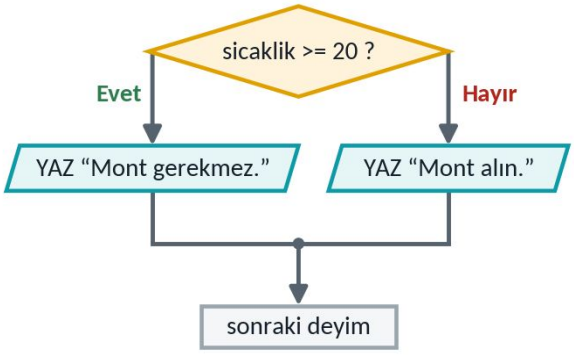
ÇIKTI (GERÇEK ÇALIŞTIRMA; SON İKİ SATIR)

```
(10 > 3) = 1, (10 < 3) = 0
gecti = 1, devamsiz = 0, sizeof(bool) = 1
```

3.2 if ve if-else; blok { } ve kapsam

`if (koşul) deyim` koşul doğruysa deyimi çalıştırır, yanlışsa atlar. Birden çok deyim **blok** `{ … }` ile bağlanır; ders standardımız gövde tek satır olsa bile süslü parantezdir (Büyük Sorudaki hata bu kuralın ihlalidir). `if-else` iki yoldan **tam olarak birini** seçer; `else` koşulsuz "aksi hâlde" dalıdır, H2'deki karar sembolünün (◇) Hayır dalıdır (Şekil 5.1).

Şekil 5.1
if-else: iki yoldan biri (H05_K03_if_blok.c)



H05_K03_if_blok.c · satır 18–22

```
18  if (sicaklik >= 20) {
19      printf("Mont gerekmez.\n");
20  } else {
21      printf("Mont alın.\n");
22  }
```

İki yoldan TAM OLARAK biri çalışır.
else'in kendi koşulu yoktur:
"aksi hâlde" demektir.
Programın çıktısı: "Mont gerekmez."

H05_K03_if_blok.c · satır 24–28

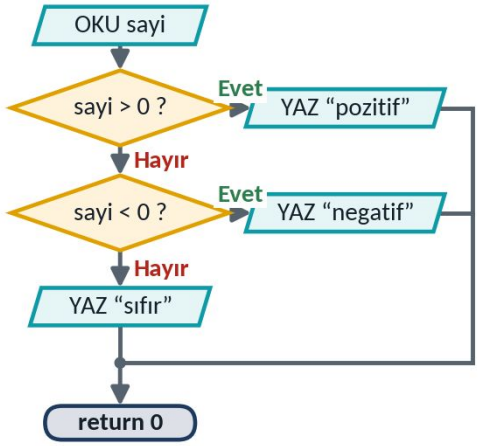
```c
    /* Blok bir kapsam (scope) oluşturur */
    if (sicaklik > 30) {
        int fark = sicaklik - 30;        /* yalnızca bu blokta yaşar */
        printf("Eşik %d derece aşıldı.\n", fark);
    }
```

Blok bir kapsam (scope) oluşturur: blokta bildirilen `fark` bloğun dışında kullanılırsa program derlenmez: `error: 'fark' undeclared (first use in this function)` (HA9). Değeri bloktan sonra da kullanacaksanız değişkeni bloktan önce bildirin (kapsam ve ömür: H9).

3.3 else-if merdiveni ve koşulların sırası

İkiden fazla yol için `else if` **merdiveni** kurulur: koşullar yukarıdan aşağıya sınanır, **ilk doğru koşulun gövdesi çalışır, geri kalanına bakılmaz**; hiçbiri doğru değilse son `else` çalışır (Şekil 5.2). `else if` ayrı bir anahtar sözcük değildir: `else`'in gövdesi bir sonraki `if`'tir.

Şekil 5.2
else-if merdiveni: pozitif/negatif/sıfır (Örnek 1)



H05_01_isaret.c · satır 15–21

```c
if (sayi > 0) {
    printf("%d pozitiftir.\n", sayi);
} else if (sayi < 0) {
    printf("%d negatiftir.\n", sayi);
} else {
    printf("Sayı sıfırdır.\n");
}
```

Koşullar yukarıdan aşağıya sınanır; ilk doğru koşulun gövdesi çalışır, gerisine hiç bakılmaz.
Son else: "hiçbiri doğru değilse".

Merdivenin en sık kullanıldığı yer **aralıklardır**. Harf notu için Recep Tayyip Erdoğan Üniversitesinin yürürlükteki tablosunu kullanıyoruz (Tablo 5.1; Recep Tayyip Erdoğan Üniversitesi [RTEÜ], 2025, Madde 30). H2 Örnek 6'daki beş düzeyli tablo, algoritmayı öğrenmek için basitleştirilmiş bir örnekti; artık kurumun gerçek tablosu geçerlidir.

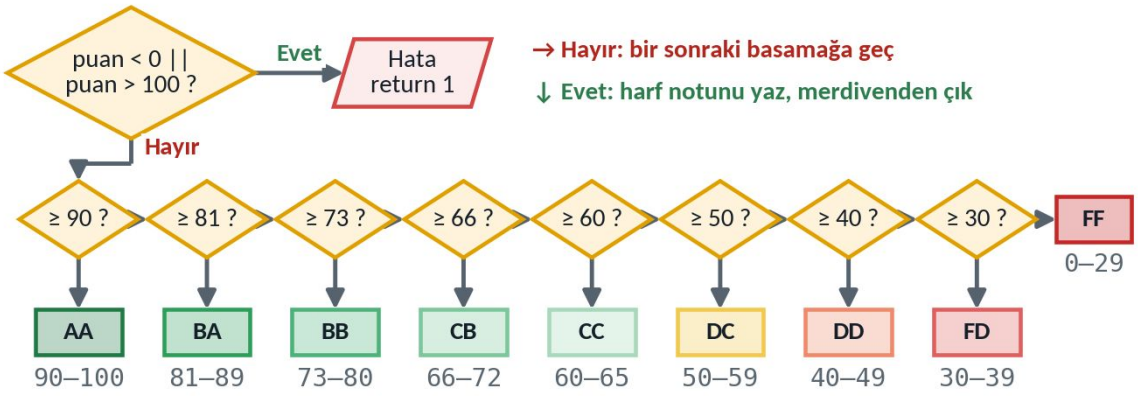
Tablo 5.1
*RTEÜ harf notu tablosu (RTEÜ, 2025, Madde 30). *DC: o yarıyılın ağırlıklı not ortalaması (YANO) en az 2,00 ise başarılı sayılır. Ara sınav %40, yarıyıl sonu sınavı %60 ağırlıklıdır; yarıyıl sonu sınavından en az 50 alınmalıdır, aksi hâlde FF verilir*

Harf	AA	BA	BB	CB	CC	DC	DD	FD	FF
Puan	90–100	81–89	73–80	66–72	60–65	50–59	40–49	30–39	0–29
Katsayı	4,0	3,5	3,0	2,5	2,0	1,5	1,0	0,5	0,0
Durum	başarılı	başarılı	başarılı	başarılı	başarılı	şartlı*	başarısız	başarısız	başarısız

Ders içi modelleme varsayımı: örneklerde puanı tamsayı alıyoruz; not ortalaması kesirli çıktığında (Mini-OBS) ortalama en yakın tamsayıya yuvarlanıp (`yuvarlanmis = (int)(ortalama + 0.5)`)

tabloya bakılır. Aralıklar **büyükten küçüğe** sıralanırsa her basamakta yalnızca alt sınırı sınamak yeter: `puan >= 81` basamağına gelindiyse puanın 90'dan küçük olduğu zaten bellidir. Sıra bozulursa program derlenir ama yanlış çalışır: `puan >= 50` önce sınanırsa 95 de "şartlı başarılı" sayılır (Alıştırma 5.3). Merdiven ve aralıkların sayı doğrusundaki yeri için bkz. Şekil 5.3 ve Şekil 5.6.

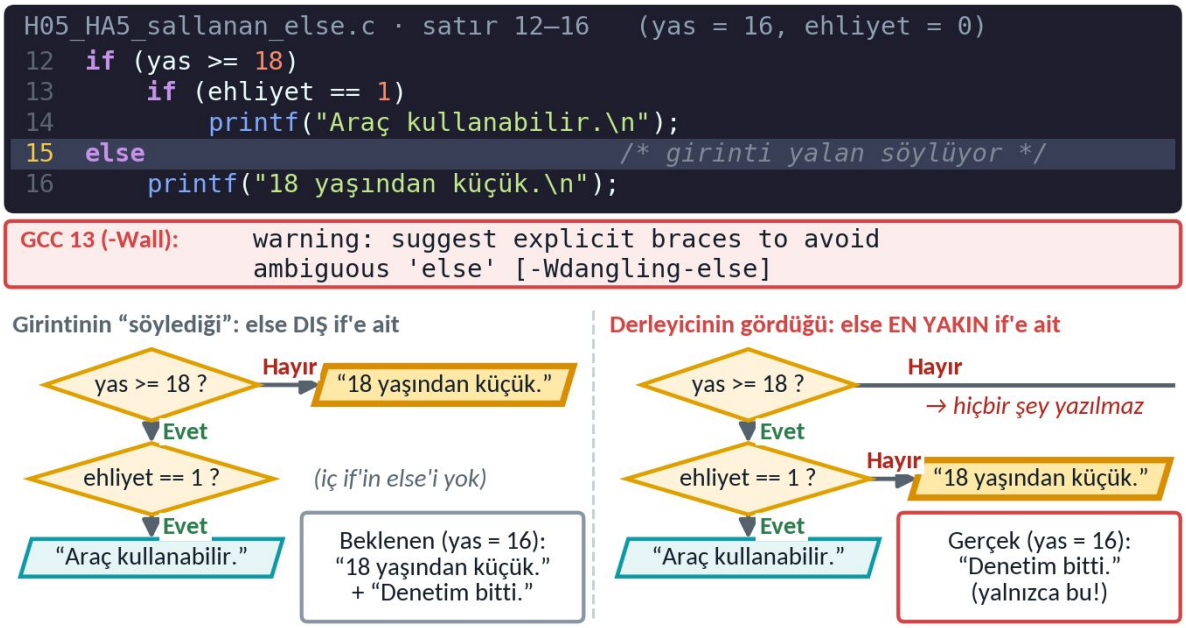
Şekil 5.3
Harf notu karar ağacı: girdi doğrulama + dokuz kademeli else-if merdiveni (Örnek 6)



3.4 İç içe if ve sallanan else

Bir `if`'in gövdesinde başka bir `if` bulunabilir (**iç içe if**): ikinci soru yalnızca ilk soru "evet" ise anlamlıdır; yaş 18'in üstündeyse ehliyete bakılır. Üç sayının en büyüğünün ilk çözümü de iç içe if'tir (Örnek 4). Süslü parantez kullanılmazsa **sallanan else** (dangling else) sorunu doğar: `else` her zaman **en yakın, henüz else'i olmayan if'e** aittir; girinti derleyici için anlamsızdır. Şekil 5.4: girinti `else`'in dıştaki `yas >= 18`'e ait olduğunu "söylüyor", derleyici ise onu içteki `ehliyet == 1`'e bağlıyor; 16 yaşındaki kullanıcıya uyarı yazılmıyor. GCC `-Wall` ile `-Wdangling-else` uyarısı verir.

Şekil 5.4
Sallanan else: girintinin söylediği ile derleyicinin gördüğü (gerçek GCC 13 uyarısı ve çıktısı)



3.5 Bileşik koşullar: doğruluk tabloları, De Morgan ve kısa devre

"Puan 0 ile 100 arasında" iki karşılaştırmanın **VE**'sidir (`puan >= 0 && puan <= 100`), "aralığın dışında" ise **VEYA**'sıdır (`puan < 0 || puan > 100`):

H05_K04_mantiksal_kosullar.c · satır 10–26

```c
    int puan = 105;

    /* Aralığın İÇİNDE mi? İki koşul birlikte: && */
    if (puan >= 0 && puan <= 100) {
        printf("%d geçerli bir puan.\n", puan);
    }

    /* Aralığın DIŞINDA mı? Biri yeter: || */
    if (puan < 0 || puan > 100) {
        printf("%d geçersiz bir puan.\n", puan);
    }

    /* De Morgan: !(A && B) ile (!A || !B) aynı sonucu verir */
    int ic = (puan >= 0 && puan <= 100);
    int dis1 = !(puan >= 0 && puan <= 100);
    int dis2 = (puan < 0 || puan > 100);
    printf("ic = %d, dis1 = %d, dis2 = %d\n", ic, dis1, dis2);
```

Doğruluk tablosu bir ifadenin tüm girdi birleşimlerindeki değerini listeler (Şekil 5.5; değerler `H05_K05_de_morgan_tablosu.c`'nin gerçek çıktısıdır) ve **De Morgan kurallarını** gösterir: `!(A && B)` ≡ `!A || !B` ("ikisi birden değil" = "en az biri değil") ve `!(A || B)` ≡ `!A && !B` ("hiçbiri").

Şekil 5.5
&&, || doğruluk tabloları ve De Morgan eşdeğerlikleri (K05'in gerçek çıktısı)

girdiler		VE · VEYA		De Morgan 1: eşit		De Morgan 2: eşit	
A	B	A && B	A || B	!(A && B)	!A || !B	!(A || B)	!A && !B
0	0	0	0	1	1	1	1
0	1	0	1	1	1	0	0
1	0	0	1	1	1	0	0
1	1	1	1	0	0	0	0

De Morgan: !(A && B) ≡ !A || !B ve !(A || B) ≡ !A && !B (dört satırın hepsinde eşit)

De Morgan olumsuz bir koşulu `!` kullanmadan yazmayı sağlar: `&&` ↔ `||` yer değiştirir, her karşılaştırma tersine döner (`>=` → `<`, `<=` → `>`, `==` → `!=`): `!(puan >= 0 && puan <= 100)` → `puan < 0 || puan > 100`. Çıktıdaki `dis1 = 1, dis2 = 1` iki yazımın eşitliğini doğrular (Örnek 11'deki `!ogrenci_mi && !hoca_mi` de bir uygulamadır).

Kısa devre değerlendirme (H4): `&&`'nin solu 0 ise sağ taraf **hiç hesaplanmaz**; `||`'nin solu doğruysa da öyle. Bu bir güvenlik aracıdır: `adet != 0 && toplam / adet > 10` koşulunda sıfıra bölme asla gerçekleşmez; ters sırada yazılsaydı tanımsız davranış oluşurdu.

H05_K04_mantiksal_kosullar.c · satır 29–35

```c
    int toplam = 50;
    int adet = 0;
    if (adet != 0 && toplam / adet > 10) {
        printf("Ortalama 10'dan büyük.\n");
    } else {
        printf("Ortalama hesaplanmadı (adet = 0).\n");
    }
```

ÇIKTI (GERÇEK ÇALIŞTIRMA)

```
$ ./mantiksal
105 geçersiz bir puan.
ic = 0, dis1 = 1, dis2 = 1
Ortalama hesaplanmadı (adet = 0).
```

3.6 Koşul operatörü ?:

`koşul ? değer1 : değer2` C'nin tek üç işlenenli operatörüdür: koşul doğruysa değeri `değer1`, yanlışsa `değer2`'dir. `?:` bir **ifadedir**; atamanın sağında ya da `printf` argümanında kullanılır (`%s` tırnak içindeki metni yazdırır; H13).

H05_K06_kosul_operatoru.c · satır 10–19

```c
    int n = -7;

    /* koşul ? doğruysa_değer : yanlışsa_değer */
    int mutlak = (n < 0) ? -n : n;
    printf("|%d| = %d\n", n, mutlak);

    printf("%d bir %s sayıdır.\n", n, (n % 2 == 0) ? "çift" : "tek");

    int final_notu = 45;
    printf("Final barajı: %s\n", (final_notu >= 50) ? "geçildi" : "FF");
```

ÇIKTI (GERÇEK ÇALIŞTIRMA)

```
$ ./kosul_op
|-7| = 7
-7 bir tek sayıdır.
Final barajı: FF
```

`?:`'yi **değer seçmek** için kullanın; iki yolda farklı işler yapılacaksa `if-else` daha okunaklıdır. İç içe `?:` zincirlerinden kaçının.

3.7 switch-case: case, break, default ve bilinçli düşüş

Bir tamsayı ifadesinin **ayrık sabit değerlerine** göre dallanmak için `switch` kullanılır (menü, işlem karakteri, ay). İfade bir kez hesaplanır ve **eşleşen case etiketine atlanır**; eşleşme yoksa `default`'a gidilir (`H05_K07_switch_ders_programi.c`: 1 → "Pazartesi…", 9 → default). Case etiketleri yalnızca **giriş noktalarıdır**: akış `break` ile switch'in sonuna atlanana kadar alt alta sürer (Örnek 9'un akış diyagramı sunumdadır).

- Case etiketi **tamsayı sabit ifadesidir** (`3`, `'+'`); değişken, `double` ya da aralık (`case 90..100`) C17'de yazılamaz (ISO/IEC, 2018). Aynı değerli iki etiket derleme hatasıdır; `default` her zaman yazılmalıdır.
- `break` unutulursa akış alttaki case'e **düşer** (fall-through); çoğu zaman hatadır (HA7). GCC `-Wextra` ile "this statement may fall through" uyarısı verir (`-Wimplicit-fallthrough`).

Düşüş **bilinçli** de olabilir: (1) **art arda case etiketleri** (`case 6: case 7:`) aynı gövdeyi paylaşır ve GCC uyarı vermez (Örnek 10); (2) **gövdeli düşüşte** her kademe bir öncekinin yetkilerini de kapsar; niyet `/* fall through */` yorumuyla belirtilir; GCC yorumu tanır ve uyarıyı susturur: aşağıdaki program `-Wall -Wextra` altında sıfır uyarıyla derlenir, seviye 3 için üç menünün hepsi açılır. C23'teki `[[fallthrough]];` özniteliği aynı işi görür; biz C17 kullanıyoruz.

H05_K08_yetki_kademeleri.c · satır 16–29

```
16      switch (seviye) {
17      case 3:
18          printf("  - Kullanıcı yönetimi\n");
19          /* fall through */
20      case 2:
21          printf("  - Not girişi\n");
22          /* fall through */
23      case 1:
24          printf("  - Not görüntüleme\n");
25          break;
26      default:
27          printf("  (hiçbiri: geçersiz seviye)\n");
28          break;
29      }
```

ÇIKTI (GERÇEK ÇALIŞTIRMA)

```
$ ./yetki
Yetki seviyesi (1-3): 3
Açılan menüler:
  - Kullanıcı yönetimi
  - Not girişi
  - Not görüntüleme
```

3.8 Girdi doğrulama: savunmacı programlama

H3'teki VKİ programı boy 0 için `inf`, harf girildiğinde `-nan` yazıyordu. **Savunmacı programlama** girdiyi kullanmadan önce doğrular: (1) okuma başarılı mı (`scanf` 1 döndürdü mü; H3 Ek örnek 13), (2) değer aralıkta mı? Geçersizse açıklayıcı ileti yazılır ve `return 1;` ile çıkılır; 0 dışı dönüş değeri işletim sistemine "hata ile bitti" der (kabukta `$?`).

H05_K09_girdi_dogrulama.c · satır 12–21

```c
    printf("Puan (0-100): ");
    if (scanf("%d", &puan) != 1) {          /* 1) Sayı mı? */
        printf("Hata: bir tamsayı girmelisiniz.\n");
        return 1;
    }
    if (puan < 0 || puan > 100) {           /* 2) Aralıkta mı? */
        printf("Hata: %d aralık dışında (0-100).\n", puan);
        return 1;
    }
    printf("Kabul edildi: %d\n", puan);      /* 3) Artık güvenle kullan */
```

ÇIKTI (GERÇEK ÇALIŞTIRMA)

```
$ ./dogrulama      # aralık dışı
Puan (0-100): 105
Hata: 105 aralık dışında (0-100).
[çıkış kodu: 1]

$ ./dogrulama      # sayı değil
Puan (0-100): abc
Hata: bir tamsayı girmelisiniz.
[çıkış kodu: 1]
```

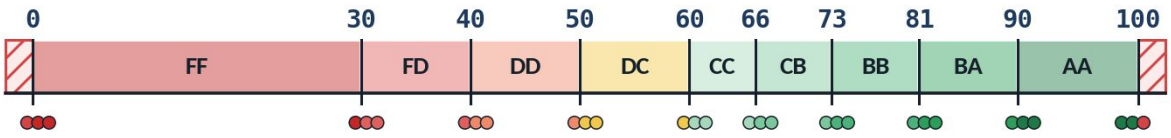
> NOT · **"Geçersizse tekrar sor" neden bu hafta yok?**
>
> Okuma adımını **tekrarlamak** bir döngüdür; karar yapılarıyla ancak "kabul et ya da sonlandır" diyebiliriz. H6'da `while`/`do-while` ile "geçerli değer girilene kadar sor" desenini yazacağız (Mini-OBS v0.6).

3.9 Test tasarımı: dal kapsamı ve sınır değer çözümlemesi

Karar yapısıyla program bir **yollar ağı** olur; tek bir girdiyle denemek yolların çoğunu hiç çalıştırmamaktır. İki teknik kullanacağız:

- **Dal kapsamı** (branch coverage): her dal en az bir testle çalıştırılır (harf notunda 9 harf + 2 geçersiz dal; Örnek 6'nın 12 çalıştırması).
- **Sınır değer çözümlemesi** (boundary value analysis): hatalar sınırlarda saklanır (`>` yerine `>=`). Her sınır için **üç test**: bir altı, kendisi, bir üstü (59, 60, 61); geçerli aralığın uçları ve hemen dışı (−1, 101) da test edilir.
- **Beklenen** çıktı programa bakmadan **tablodan** yazılır, sonra **gerçek** çıktıyla karşılaştırılır (✓/✗); beklenen değeri programdan kopyalamak hatayı onaylamaktır.

Şekil 5.6
Harf notu aralıkları, sınırlar ve sınır değer testleri (Örnek 13'ün 30 testi)



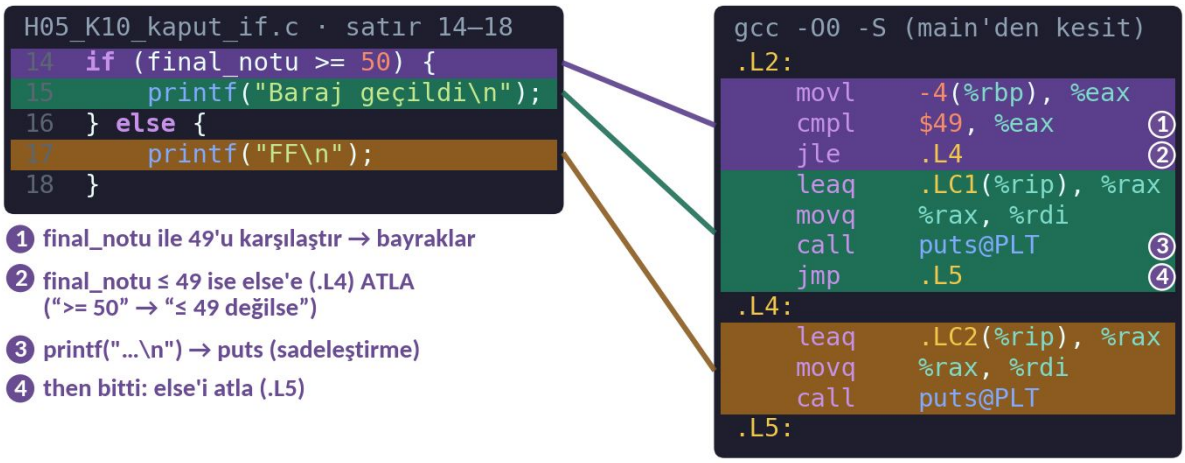
Sınıflar [alt, üst] kapalı aralık; taralı uçlar geçersiz (< 0, > 100); nokta rengi = verilen harf notu.

- Her sınırda üç test: bir altı · kendisi · bir üstü (ör. 59 · 60 · 61; toplam 30 test, Örnek 13).

3.10 Kaputun Altında: kararlar makine dilinde

İşlemcide "if" komutu yoktur. Derleyici her kararı **karşılaştır** (`cmp`: iki değeri çıkarır, sonucu saklamaz, **bayrakları** ayarlar) ve **koşullu atla** (`jle`, `jg`, `je`…: bayraklara göre program sayacını başka adrese taşır) çiftine çevirir (H2'deki sezgi). Çıktılar `gcc -S` ile üretildi; Ubuntu'nun yığın koruyucu ve CET komutları kapatıldı (varsayılan bayraklı `.s` dosyaları da `kodlar/ciktilar/` altında).

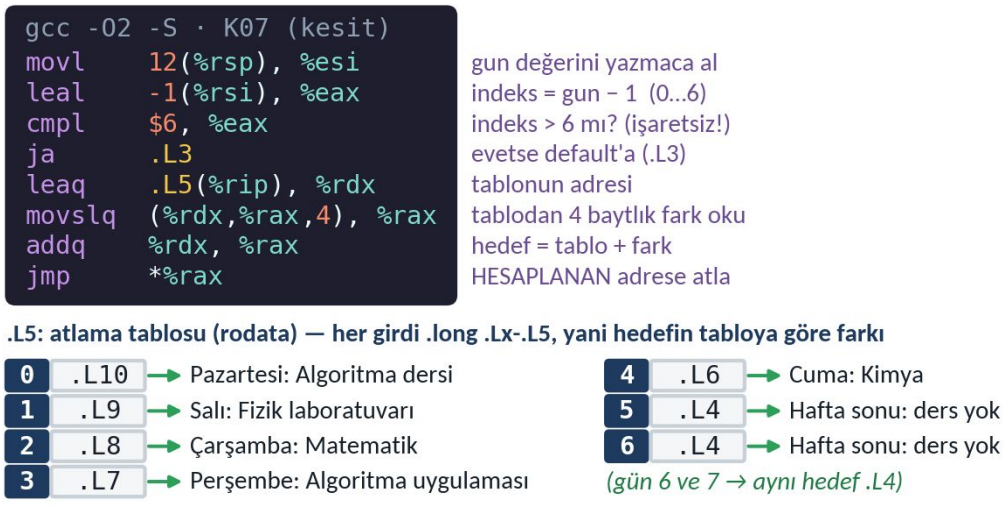
Şekil 5.7
if (final_notu >= 50) → cmpl $49 + jle: aynı renk aynı C satırının komutları (gcc -O0 -S; yığın koruyucu ve CET kapalı; gerçek çıktı: kodlar/ciktilar/H05_K10_kaput_if_O0.s)



> **KAPUTUN ALTINDA · Neler görüyoruz?**
>
> - `cmpl $49, %eax` + `jle .L4`: derleyici `final_notu >= 50` koşulunu (final barajı) **tersine çevirmiştir** ("≤ 49 ise else'e atla"); atlama olmazsa then dalı çalışır, sonundaki `jmp .L5` else'in üzerinden atlar: if-else'in makinedeki kalıbı; `-O2`'de de aynıdır (`cmpl $49, 12(%rsp)` + `jle .L3`). `printf("Baraj geçildi\n")` ise `puts`'a sadeleştirilmiştir.

Atlama tablosu. Case değerleri **yoğunsa** (1…7) derleyici ayrı karşılaştırmalar yerine bir **atlama tablosu** (jump table) kurar: değerden indeks hesaplar, tablodan hedef adresi okur ve **dolaylı** atlar (Şekil 5.8; tam dosya `kodlar/ciktilar/H05_K07_switch_ders_programi_O2.s`).

Şekil 5.8
Yoğun case değerleri → atlama tablosu (-O2, gerçek çıktı)



Gün 6 ile 7 aynı etikete (`.L4`) gider: art arda case etiketlerinin makinedeki karşılığı. `cmpl $6` + `ja` tek **işaretsiz** karşılaştırmayla iki sınırı birden denetler (gün ≤ 0 ise `gun - 1` işaretsiz yorumda çok büyüktür). Değerler **seyrekse** (1, 100, 1000, 10000, 100000) GCC değerleri ortadan bölen bir **karşılaştırma ağacı** üretir (`H05_K11`; her yolda en fazla üç `cmpl`).

Koşullu taşıma ve dal tahmini. `int buyuk = (a > b) ? a : b;` için GCC bu makinede `-O0`'da bile atlama değil **koşullu taşıma** üretti: `cmovge %edx, %eax` (`H05_K12`). İşlemciler komutları bir **hat** (pipeline) üzerinde üst üste işler; koşullu atlamanın sonucu belli olmadan sonraki komut getirilmelidir. İşlemci bu yüzden **dal tahmini** (branch prediction) yapar: dalın geçmişine bakarak bir yol seçer; tahmin yanlışsa yanlış yoldan getirilenler atılır ve hat yeniden doldurulur. cmov'da atlama olmadığından tahmin de gerekmez. Bu hafta bu yalnızca bir sezgidir.

Hangi yapıyı seçmeli? Tek dal → `if`; iki yoldan birinde **değer** seçiliyorsa `?:`, işler seçiliyorsa `if-else`; aralıklar → else-if merdiveni; ayrık tamsayı sabitleri → `switch`.

4. Çözümlü Örnekler: Örnek Merdiveni

Her basamak bir öncekine yalnızca bir yeni öge ekler (Tablo 5.2). Önce çıktıyı **tahmin edin**, sonra çalıştırın (PRIMM). Tüm çıktılar GCC 13 / x86-64 Linux'ta gerçekten alınmıştır (`kodlar/ciktilar/`); kullanıcı girdisi istemin sonundadır, `[çıkış kodu: 1]` programın `return 1` ile bittiğini gösterir. **Amiral örnekler** (6 ve 12) dokuz adımlı şablonla işlenmiştir.

Tablo 5.2
Haftanın örnek merdiveni (dosyalar: kodlar/H05_01_... – H05_13_...)

#	Örnek	Yeni öge / amaç	Düzey
1	Pozitif / negatif / sıfır	if – else if – else	★
2	Sayının tek/çift olması	if-else ve %	★
3	İki sayının büyüğü	Karşılaştırma; ?: ile kısa yazım	★
4	Üç sayının en büyüğü (üç farklı çözüm)	İç içe if, && ve ardışık güncellemenin karşılaştırılması	★★
5	Artık yıl (if zinciri ve tek ifade)	Bileşik koşul; okunabilirlik tartışması	★★
6	Puandan harf notu	else-if merdiveni; aralık sıralaması	★★
7	Üçgen geçerliliği ve türü	Çok koşullu mantık; eşkenar/ikizkenar/çeşitkenar/dik	★★
8	İkinci dereceden denklemin kökleri	Δ > 0, Δ = 0, Δ < 0 dalları	★★
9	Dört işlem hesap makinesi	switch; sıfıra bölme denetimi	★★
10	Ayın gün sayısı	switch'te bilinçli fall-through	★★
11	PIN doğrulama ve kullanıcı yetkisi	Mantıksal operatörler; güvenlik bakışı	★★
12	Kademeli elektrik faturası/vergi dilimi	Aralıklı hesaplama; sınır değerler	★★★
13	Harf notu için sınır değer test tablosu	Test tasarımı (49/50/51 türü değerler)	★★★

4.1 Örnek 1 ★ Pozitif / negatif / sıfır

ÇIKTI (GERÇEK ÇALIŞTIRMA)

```
$ ./isaret      # pozitif
Bir tamsayı girin: 7
7 pozitiftir.

$ ./isaret      # sıfır
Bir tamsayı girin: 0
Sayı sıfırdır.
```

Kodun karar kısmı dosya satır numaralarıyla Şekil 5.2 içindedir. **Yeni öge:** `if – else if – else`; üç test üç dalı birer kez çalıştırır (dal kapsamı). **Değiştir:** "sıfır" dalını en üste taşıyın; çıktı değişir mi?

4.2 Örnek 2 ★ Sayının tek/çift olması

H05_02_tek_cift.c · satır 15–20

```
15      if (sayi % 2 == 0) {
16          printf("%d çifttir.\n", sayi);
17      } else {
18          printf("%d tektir.\n", sayi);
19      }
20      printf("(%d %% 2 = %d)\n", sayi, sayi % 2);
```

ÇIKTI (GERÇEK ÇALIŞTIRMA)

```
$ ./tek_cift      # negatif tek: kalan -1
Bir tamsayı girin: -7
-7 tektir.
(-7 % 2 = -1)
```

Yeni öge: `if-else` ve `%`. −7 için kalan **−1**'dir (C99'dan beri bölme sıfıra doğru keser; kalan bölünenin işaretini alır, H4). Bu yüzden `sayi % 2 == 1` negatif tek sayılarda **yanlış** sonuç verir; `== 0` ile çiftlik sınaması her tamsayıda doğrudur.

4.3 Örnek 3 ★ İki sayının büyüğü (if-else ve ?:)

H2 Örnek 3'te akış diyagramını çizdiğimiz (`H02_03`) kararı iki yolla yazıyoruz: satır 17–22'de H2'deki gibi `if-else` ile, aşağıda `?:` ile; satır 29–31 eşitliği ayrıca bildirir.

H05_03_iki_sayi_buyuk.c · satır 25–27

```
25      /* 2. yol: koşul operatörü ?: */
26      int buyuk2 = (a > b) ? a : b;
27      printf("?: ile      : %d\n", buyuk2);
```

ÇIKTI (GERÇEK ÇALIŞTIRMA)

```
$ ./buyuk      # eşit
İki tamsayı girin: 8 8
if-else ile : 8
?: ile      : 8
Not: iki sayı eşit.
```

Yeni öge: `?:` ile if-else'in dört satırı tek satıra iner. Eşit sayılarda `a > b` yanlıştır ve `b` seçilir; değer aynı olduğundan sonuç doğrudur.

4.4 Örnek 4 ★★ Üç sayının en büyüğü: üç çözüm

H2 Örnek 5'teki iki stratejiye (iç içe karar, ardışık güncelleme; `H02_05`) `&&` ile üçüncü bir çözüm ekleyip üçünü aynı girdilerle deniyoruz. İç içe çözüm (satır 17–31) Bölüm 3.4'teki yapıdır; ardışık güncelleme (satır 43–50) H2'dekiyle aynıdır; yeni olan `&&` çözümü:

H05_04_uc_sayi_en_buyuk.c · satır 33–41

```
33      /* Çözüm 2: && ile bileşik koşullar */
34      int eb2 = 0;
35      if (a >= b && a >= c) {
36          eb2 = a;
37      } else if (b >= a && b >= c) {
38          eb2 = b;
39      } else {
40          eb2 = c;
41      }
```

Tablo 5.3
Örnek 4'ün test tablosu: iç içe / && / ardışık güncelleme (gerçek çalıştırmalar)

Girdi	Beklenen	Gerçek (3 çözüm)		Girdi	Beklenen	Gerçek (3 çözüm)	
4 9 7	9	9 / 9 / 9	✓	9 4 7	9	9 / 9 / 9	✓
4 7 9	9	9 / 9 / 9	✓	9 9 2	9	9 / 9 / 9	✓
5 5 5	5	5 / 5 / 5	✓	-3 -8 -1	-1	-1 / -1 / -1	✓

Üç çözüm tüm testlerde aynı sonucu verir; fark **okunabilirlik** ve **karşılaştırma sayısındadır**: iç içe çözüm her zaman 2, `&&` çözümü en kötü durumda 4, ardışık güncelleme her zaman 2 karşılaştırma yapar ve N sayıya kolayca genişler (H6: döngüyle "en büyüğü bul"). Eşit ve negatif değerleri test etmeyi unutmayın (`>` / `>=` seçimi).

4.5 Örnek 5 ★★ Artık yıl: if zinciri ve tek ifade

H05_05_artik_yil.c · satır 17–29

```
17      bool artik1 = false;
18      if (yil % 400 == 0) {
19          artik1 = true;
20      } else if (yil % 100 == 0) {
21          artik1 = false;
22      } else if (yil % 4 == 0) {
23          artik1 = true;
24      } else {
25          artik1 = false;
26      }
27
28      /* 2. yol: tek bileşik ifade (H4 Örnek 8) */
29      bool artik2 = (yil % 4 == 0 && yil % 100 != 0) || yil % 400 == 0;
```

ÇIKTI (GERÇEK ÇALIŞTIRMA)

```
$ ./artik_yil      # 100'e bölünür, 400'e bölünmez
Yıl girin: 1900
if zinciri : artık yıl değil
tek ifade  : artık yıl değil

$ ./artik_yil      # 400'e bölünür
Yıl girin: 2000
if zinciri : artık yıl
tek ifade  : artık yıl
```

Yeni ögeler: `bool`, `true`, `false`; bileşik koşul. Zincir kuralları **en özelden genele** sıralar (400 → 100 → 4); tek ifade H4 Örnek 8'dir. Dört test (2024, 1900, 2000, 2026) zincirin dört dalını çalıştırır. Zincir kuralın "istisnaların istisnası" yapısını adım adım gösterir; tek ifade kısadır ama parantez hatasına açıktır. Sıra bozulursa (önce `% 4`) 1900 yanlışlıkla artık yıl olur.

4.6 Örnek 6 ★★ Puandan harf notu (amiral örnek)

1. Problem. 0–100 arası tamsayı puanı okuyup harf notunu (Tablo 5.1) yazdıralım: H2 Örnek 6'nın kurum tablosuna göre dokuz kademeli ve girdi doğrulamalı hâli. **2. Çözümleme.** Aralıklar bitişik ve örtüşmezdir; her geçerli puan tam olarak bir harfe düşer; sınırlar tamsayılar arasındadır (59 → DC, 60 → CC); 0–100 dışı reddedilir.

Tablo 5.4
Örnek 6'nın IPO çözümlemesi

GİRDİ	İŞLEM	ÇIKTI
• `puan`: tamsayı (0–100)	• Geçersizse ileti + `return 1` • Aralıkları büyükten küçüğe sına	• Harf notu (9 kademeden biri)

3. Algoritma. Karar ağacı için bkz. Şekil 5.3: önce doğrulama, sonra `puan >= 90`, `puan >= 81`, … `puan >= 30` basamakları; hiçbiri değilse FF. **4. C kodu:**

H05_06_harf_notu.c · satır 15–39 `gcc -std=c17 -Wall -Wextra -pedantic -g H05_06_harf_notu.c -o harf_notu`

```
15      if (puan < 0 || puan > 100) {                /* girdi doğrulama */
16          printf("Hata: %d geçersiz; puan 0-100 olmalı.\n", puan);
17          return 1;
18      }
19
20      printf("Harf notu: ");
21      if (puan >= 90) {
22          printf("AA\n");
23      } else if (puan >= 81) {
24          printf("BA\n");
25      } else if (puan >= 73) {
26          printf("BB\n");
27      } else if (puan >= 66) {
28          printf("CB\n");
29      } else if (puan >= 60) {
30          printf("CC\n");
31      } else if (puan >= 50) {
```

```
32          printf("DC\n");
33      } else if (puan >= 40) {
34          printf("DD\n");
35      } else if (puan >= 30) {
36          printf("FD\n");
37      } else {
38          printf("FF\n");
39      }
```

Satır 10–13 puanı okur (tam dosya: `kodlar/H05_06_harf_notu.c`).

5. Satır satır açıklama.

- **15. satır:** Girdi doğrulama (De Morgan ile "0 ≤ puan ≤ 100 değil"); geçersizse `return 1`, merdivene girilmez.
- **21–31. satırlar:** Sekiz basamak; her biri yalnızca alt sınırı sınar (üst sınırı bir önceki basamağın "yanlış"ı güvenceler). Son `else` 0–49'dur: doğrulamadan geçen her puan bir dala düşer.

6. İzleme tablosu (girdi 81: BB/BA sınırı):

Tablo 5.5
Örnek 6'nın izleme tablosu (girdi 81); 7. adım: return 0 — merdivenin kalanına bakılmaz

Adım	Satır	puan	Koşul	Değer	Çıktı
1	13	**81**	—	—	
2	15	81	`puan < 0 || puan > 100`	0	
3	20	81	—	—	`Harf notu:`
4	21	81	`puan >= 90`	**0**	
5	23	81	`puan >= 81`	**1**	
6	24	81	—	—	**BA**

7. Örnek çalıştırma:

ÇIKTI (GERÇEK ÇALIŞTIRMA)

```
$ ./harf_notu      # geçersiz (üst)
Puanı girin (0-100): 101
Hata: 101 geçersiz; puan 0-100 olmalı.
[çıkış kodu: 1]
```

8. Test tablosu (her dal için bir test; gerçek çalıştırmalar):

Tablo 5.6
Örnek 6'nın dal kapsamı test tablosu (11 dalın tamamı; gerçek çalıştırmalar)

Girdi	Beklenen	Gerçek		Girdi	Beklenen	Gerçek	
81	BA	BA	✓	95	AA	AA	✓
85	BA	BA	✓	77	BB	BB	✓
69	CB	CB	✓	62	CC	CC	✓

Girdi	Beklenen	Gerçek		Girdi	Beklenen	Gerçek	
55	DC	DC	✓	45	DD	DD	✓
35	FD	FD	✓	20	FF	FF	✓
101	Hata	Hata (kod 1)	✓	-5	Hata	Hata (kod 1)	✓

Bu tablo her dalı çalıştırır ama sınırları sistematik sınamaz: `>= 81` yerine `> 81` yazılsaydı yalnızca 81 testi hatayı yakalardı; sınırların sistematik testi Örnek 13'tedir. **9. Değiştir – genişlet.** (a) durum satırı ekleyin (başarılı / şartlı başarılı / başarısız). (b) Puanı `double` okuyun (89.5): hangi sınırlar belirsizleşir? (Mini-OBS'teki `yuvarlanmis` yanıtıdır.)

4.7 Örnek 7 ★★ Üçgen geçerliliği ve türü

Problem. Üç kenar uzunluğu verilen üçgenin geçerli olup olmadığını, kenarlarına göre türünü ve dik olup olmadığını bulalım: önce geçerlilik, sonra kenarlara göre tür, en son dik açı denetimi.

H05_07_ucgen.c · satır 27–41

```
27      /* 2) Kenarlara göre tür */
28      if (a == b && b == c) {
29          printf("Eşkenar üçgen\n");
30      } else if (a == b || b == c || a == c) {
31          printf("İkizkenar üçgen\n");
32      } else {
33          printf("Çeşitkenar üçgen\n");
34      }
35
36      /* 3) Açıya göre: dik mi? (Pisagor; tamsayılarla tam sonuç) */
37      if (a * a + b * b == c * c || a * a + c * c == b * b
38          || b * b + c * c == a * a) {
39          printf("Ayrıca dik üçgendir.\n");
40      }
41      return 0;
```

Tablo 5.7
Örnek 7'nin test tablosu (gerçek çalıştırmalar; ✓ = beklenen sınıfla aynı)

Girdi	Gerçek çıktı		Girdi	Gerçek çıktı	
3 4 5	Çeşitkenar üçgen / Ayrıca dik üçgendir.	✓	5 5 5	Eşkenar üçgen	✓
5 5 8	İkizkenar üçgen	✓	4 6 7	Çeşitkenar üçgen	✓
5 13 12	Çeşitkenar üçgen / Ayrıca dik üçgendir.	✓	1 2 3	Geçersiz: üçgen eşitsizliği sağlanmıyor.	✓
0 4 5	Geçersiz: kenarlar pozitif olmalı.	✓			

Yeni ögeler: çok koşullu mantık. Önce geçerlilik denetlenir (bir kenar ≤ 0 ise ya da `a + b <= c || a + c <= b || b + c <= a` ise `return 1`), sonra tür. Eşkenar denetimi iki karşılaştırmanın VE'sidir (`a == b && b == c`); `a == b == c` başka bir şey demektir (HA6). "1 2 3" dejenere bir sınır durumudur (`<=` reddeder); "5 13 12" hipotenüsün her zaman `c` olmadığını gösterir. Pisagor denetimi tamsayılarla yapıldığı için `==` güvenlidir.

4.8 Örnek 8 ★★ İkinci dereceden denklemin kökleri

H05_08_ikinci_derece.c · satır 23–38

```c
    int delta = b * b - 4 * a * c;      /* tamsayı: == 0 güvenle sınanır */
    printf("Delta = %d\n", delta);

    if (delta > 0) {
        double x1 = (-b + sqrt(delta)) / (2.0 * a);
        double x2 = (-b - sqrt(delta)) / (2.0 * a);
        printf("İki farklı gerçel kök: x1 = %.4f, x2 = %.4f\n", x1, x2);
    } else if (delta == 0) {
        double x = -b / (2.0 * a);
        printf("Çakışık (çift) kök: x = %.4f\n", x);
    } else {
        double gercel = -b / (2.0 * a);
        double sanal = sqrt(-delta) / fabs(2.0 * a);
        printf("Gerçel kök yok; karmaşık kökler: %.4f ± %.4fi\n",
               gercel, sanal);
    }
```

ÇIKTI (GERÇEK ÇALIŞTIRMA)

```
$ ./denklem      # Δ > 0
a, b, c katsayıları (tamsayı): 1 -3 2
Delta = 1
İki farklı gerçel kök: x1 = 2.0000, x2 = 1.0000

$ ./denklem      # Δ < 0
a, b, c katsayıları (tamsayı): 1 0 4
Delta = -16
Gerçel kök yok; karmaşık kökler: 0.0000 ± 2.0000i
```

H4 Örnek 11'de "H5'e köprü" olarak bıraktığımız Δ < 0 durumu artık bir dal. **Yeni öge:** matematiksel durum çözümlemesinin dallara aktarılması (`a = 0` daha önce elenir: satır 18–21). Katsayılar tamsayı olduğundan `delta == 0` tamdır; `double` bir Δ ile güvenilmezdi (HA8). `sqrt` ve `fabs` için `-lm` gerekir.

4.9 Örnek 9 ★★ Dört işlem hesap makinesi (switch)

H05_09_hesap_makinesi.c · satır 27–34

```c
    case '/':
        if (b == 0) {                       /* sıfıra bölme denetimi */
            printf("Hata: sıfıra bölme tanımsızdır.\n");
        } else {
            printf("%d / %d = %.2f\n", a, b, (double)a / b);
        }
        break;
    default:
```

ÇIKTI (GERÇEK ÇALIŞTIRMA)

```
$ ./hesap      # sıfıra bölme
İşlem girin (ör. 12 / 4): 5 / 0
Hata: sıfıra bölme tanımsızdır.
```

Yeni ögeler: karakter sabitleriyle `switch` ve case içinde `if` (sıfıra bölme denetimi; tamsayı sıfıra bölme **tanımsız davranıştır**, x86-64'te genellikle SIGFPE ile çöker). `"%d %c %d"` biçimindeki `%c` öncesindeki boşluk sayıdan sonraki boşlukları atlar. Diğer çalıştırmalar: `12 / 4 = 3.00`, `7 / 2 = 3.50`, `8 ^ 2` → "Bilinmeyen işlem".

4.10 Örnek 10 ★★ Ayın gün sayısı: bilinçli düşüş

H05_10_ay_gun.c · satır 17–26

```
17      switch (ay) {
18      case 1:
19      case 3:
20      case 5:
21      case 7:
22      case 8:
23      case 10:
24      case 12:
25          gun = 31;            /* yedi etiket, tek gövde */
26          break;
```

ÇIKTI (GERÇEK ÇALIŞTIRMA)

```
$ ./ay_gun      # şubat, artık yıl
Ay ve yıl girin (ör. 2 2024): 2 2024
2. ay (2024) 29 gün çeker.
```

Yeni öge: art arda case etiketleri. 1 girildiğinde akış `case 1:`'e atlar ve deyimsiz etiketlerin üzerinden `gun = 31;` satırına düşer: bilinçli düşüş, uyarı yok. Şubat için Örnek 5'in kuralı case içinde `if-else` ile uygulanır. Diğer çalıştırmalar: 2 2026 → 28, 4 2026 → 30, 12 2026 → 31, 13 → hata ve çıkış kodu 1.

4.11 Örnek 11 ★★ PIN doğrulama ve kullanıcı yetkisi

H05_11_pin_yetki.c · satır 25–31

```
25      bool ogrenci_mi = (kullanici == OGRENCI_NO && pin == OGRENCI_PIN);
26      bool hoca_mi = (kullanici == HOCA_NO && pin == HOCA_PIN);
27
28      if (!ogrenci_mi && !hoca_mi) {   /* De Morgan: !(ogrenci_mi || hoca_mi) */
29          printf("Erişim reddedildi: kullanıcı no veya PIN hatalı.\n");
30          return 1;
31      }
```

ÇIKTI (GERÇEK ÇALIŞTIRMA)

```
$ ./pin      # başkasının PIN'i
Kullanıcı no: 1001
PIN         : 7391
Erişim reddedildi: kullanıcı no veya PIN hatalı.
[çıkış kodu: 1]
```

Yeni ögeler: mantıksal operatörlerle kimlik denetimi. `&&`, numara ile PIN'in **aynı hesaba** ait olmasını zorunlu kılar (1001 + öğretim üyesinin PIN'i reddedilir); ret koşulu De Morgan'dır: `!ogrenci_mi && !hoca_mi` ≡ `!(ogrenci_mi || hoca_mi)`. İleti hangi bilginin yanlış olduğunu **söylemez**; söylemek saldırgana bilgi verir.

MÜHENDİS GÖZÜYLE · **Güvenlik bakışı: bu kod yalnızca eğitseldir**

- Gerçek sistemlerde PIN ya da parola **kaynak koda yazılmaz**: derlenmiş dosyada düz metin olarak durur ve `strings` gibi araçlarla okunabilir.
- Parolanın kendisi değil, tek yönlü bir **özet** (hash) fonksiyonuyla hesaplanmış değeri saklanır ve karşılaştırılır; parola özetleri için bilerek yavaşlatılmış algoritmalar kullanılır.
- Deneme sayısı **sınırlanır** (ör. üç yanlış denemede hesabı kilitle). "Deneme sayısını say, sınıra ulaşınca dur" bir döngüdür: H6'da ekleyeceğiz.

4.12 Örnek 12 ★★★ Kademeli elektrik faturası (amiral örnek)

1. Problem. Kademeli bir tarifede (değerler örnektir) ilk 150 kWh 2,00 TL/kWh, 151–300 arası kısım 3,00 TL/kWh, 300'ü aşan kısım 4,50 TL/kWh'dir; faturayı hesaplayalım. **2. Çözümleme.** Üst kademenin fiyatı **yalnızca o kademeye düşen kısma** uygulanır (gelir vergisi dilimleri gibi); fatura tüketimle **sürekli** artar. Girdi tamsayı kWh, negatif olamaz; sınırlar 0, 150/151, 300/301.

Tablo 5.8
Örnek 12'nin IPO çözümlemesi

GİRDİ	İŞLEM	ÇIKTI
• `tuketim` (kWh, tamsayı)	• Negatifse reddet • Kademeyi bul (else-if) • Her kademenin payını topla	• Kademe numarası • Bedel (TL, 2 basamak)

3. Algoritma. `tuketim ≤ 150` → bedel = tuketim × 2,00; `tuketim ≤ 300` → 150 × 2,00 + (tuketim − 150) × 3,00; aksi hâlde 150 × 2,00 + 150 × 3,00 + (tuketim − 300) × 4,50. **4. C kodu:**

H05_12_elektrik_faturasi.c · satır 10–14 `gcc -std=c17 -Wall -Wextra -pedantic -g H05_12_elektrik_faturasi.c -o fatura`

```c
    const int KADEME1_UST = 150;        /* kWh; örnek tarife */
    const int KADEME2_UST = 300;
    const double FIYAT1 = 2.00;         /* TL/kWh */
    const double FIYAT2 = 3.00;
    const double FIYAT3 = 4.50;
```

H05_12_elektrik_faturasi.c · satır 25–39

```c
    double bedel = 0.0;
    int kademe = 0;
    if (tuketim <= KADEME1_UST) {
        kademe = 1;
        bedel = tuketim * FIYAT1;
    } else if (tuketim <= KADEME2_UST) {
        kademe = 2;
        bedel = KADEME1_UST * FIYAT1 + (tuketim - KADEME1_UST) * FIYAT2;
    } else {
        kademe = 3;
        bedel = KADEME1_UST * FIYAT1 + (KADEME2_UST - KADEME1_UST) * FIYAT2
                + (tuketim - KADEME2_UST) * FIYAT3;
    }
    printf("Tüketim: %d kWh | Kademe: %d | Bedel: %.2f TL\n",
           tuketim, kademe, bedel);
```

Aradaki satırlar (16–24) tüketimi okur ve negatif değeri `return 1` ile reddeder.

5. Satır satır açıklama.

- **10–14. satırlar:** Sınırlar ve fiyatlar adlandırılmış sabitlerdir; tarife değişince yalnızca bunlar değişir.
- **27–33. satırlar:** else-if merdiveni; `<=` 150'yi 1. kademede tutar. Her dalda önceki kademelerin **tam** payı eklenir; `int * double` çarpımında int double'a dönüşür (H4).

6. İzleme tablosu (girdi 301):

Tablo 5.9
Örnek 12'nin izleme tablosu (girdi 301)

Adım	Satır	tuketim	Koşul → değer	kademe	bedel
1	18	**301**	—	0	—
2	20	301	tuketim < 0 → 0	0	—
3	27	301	tuketim <= 150 → 0	0	0.0
4	30	301	tuketim <= 300 → 0	0	0.0
5	34	301	(else dalı)	**3**	0.0
6	35–36	301	300 + 450 + 4.5	3	**754.5**

7. Örnek çalıştırma:

ÇIKTI (GERÇEK ÇALIŞTIRMA)

```
$ ./fatura
Aylık tüketim (kWh): 301
Tüketim: 301 kWh | Kademe: 3 | Bedel: 754.50 TL
```

8. Test tablosu (normal, sınır ve hatalı girdiler; beklenen değerler elle hesaplandı):

Tablo 5.10
Örnek 12'nin sınır değer test tablosu (beklenenler elle hesaplandı; gerçek çalıştırmalar)

kWh	Beklenen	Gerçek		kWh	Beklenen	Gerçek	
120	240.00	240.00 (kademe 1)	✓	0	0.00	0.00 (kademe 1)	✓
1	2.00	2.00 (kademe 1)	✓	149	298.00	298.00 (kademe 1)	✓
150	300.00	300.00 (kademe 1)	✓	151	303.00	303.00 (kademe 2)	✓
299	747.00	747.00 (kademe 2)	✓	300	750.00	750.00 (kademe 2)	✓
301	754.50	754.50 (kademe 3)	✓	450	1425.00	1425.00 (kademe 3)	✓
-1	hata	Hata (kod 1)	✓				

Hatalı yöntemde (Alıştırma 5.9; grafik sunumda ve web'de) 150 → 151 kWh geçişi faturayı 300,00 TL'den 453,00 TL'ye sıçratır ("üst dilime geçince zarar ederim" yanılgısının kaynağı); 150/151 **sınır**

testleri bunu hemen yakalar, aralık ortasından bir test (200) yakalayamaz. **9. Değiştir.** (a) Dördüncü kademe ekleyin. (b) Parada `double` yerine kuruş cinsinden `long long` neden daha güvenli?

4.13 Örnek 13 ★★★ Harf notu için sınır değer test tablosu

Problem. Örnek 6'nın merdivenini sistematik olarak test edelim. Test için program istem yazdırmaz ve sonucu tek satırda verir (`DC şartlı başarılı`); karar mantığı Örnek 6 ile aynıdır, sonuna durum (başarılı / şartlı başarılı / başarısız) eklenmiştir:

H05_13_harf_notu_test.c · satır 36–42

```
36      if (puan >= 60) {            /* CC ve üzeri: başarılı */
37          printf(" başarılı\n");
38      } else if (puan >= 50) {     /* DC: YANO >= 2,00 ise başarılı */
39          printf(" şartlı başarılı\n");
40      } else {
41          printf(" başarısız\n");
42      }
```

Test tasarımı. On sınır (0, 30, 40, 50, 60, 66, 73, 81, 90, 100) × üç test = 30 test. Beklenenler kurum tablosundan (Tablo 5.1) elle yazılıp `H05_13_sinir_testi.sh` betiğine konuldu; betik her girdiyi programa verir ve gerçek çıktıyla karşılaştırır (noktalar: Şekil 5.6).

Tablo 5.11
Örnek 13'ün 30 sınır testi (betik çıktısından; Toplam: 30 test, geçen: 30, kalan: 0)

Girdi	Beklenen	Gerçek		Girdi	Beklenen	Gerçek		Girdi	Beklenen	Gerçek	
-1	GEÇERSİZ	GEÇERSİZ	✓	50	DC şartlı başarılı	DC şartlı başarılı	✓	74	BB başarılı	BB başarılı	✓
0	FF başarısız	FF başarısız	✓	51	DC şartlı başarılı	DC şartlı başarılı	✓	80	BB başarılı	BB başarılı	✓
1	FF başarısız	FF başarısız	✓	59	DC şartlı başarılı	DC şartlı başarılı	✓	81	BA başarılı	BA başarılı	✓
29	FF başarısız	FF başarısız	✓	60	CC başarılı	CC başarılı	✓	82	BA başarılı	BA başarılı	✓
30	FD başarısız	FD başarısız	✓	61	CC başarılı	CC başarılı	✓	89	BA başarılı	BA başarılı	✓
31	FD başarısız	FD başarısız	✓	65	CC başarılı	CC başarılı	✓	90	AA başarılı	AA başarılı	✓
39	FD başarısız	FD başarısız	✓	66	CB başarılı	CB başarılı	✓	91	AA başarılı	AA başarılı	✓
40	DD başarısız	DD başarısız	✓	67	CB başarılı	CB başarılı	✓	99	AA başarılı	AA başarılı	✓
41	DD başarısız	DD başarısız	✓	72	CB başarılı	CB başarılı	✓	100	AA başarılı	AA başarılı	✓
49	DD başarısız	DD başarısız	✓	73	BB başarılı	BB başarılı	✓	101	GEÇERSİZ	GEÇERSİZ	✓

Tüm testler geçti. Bu, hatasızlığı değil **tipik sınır hatalarının yokluğunu** gösterir: >/>= karışıklığı, yanlış eşik ya da atlanmış basamak en az bir ✗ üretirdi. Aynı düşünce final barajında da geçerlidir: Alıştırma 5.4'teki `final_notu > 50` hatasını yalnızca 50 testi yakalar.

5. Hata Avcısı

Hataların hepsi gerçekten derlenip çalıştırıldı; iletiler GCC 13'ün birebir çıktısıdır (`kodlar/hata_avcisi/`, hatalı + düzeltilmiş sürümler; `gcc -std=c17 -Wall -Wextra -pedantic -g`). HA9 dışındakilerin **hepsi geçerli C**'dir: derleyici yalnızca uyarır, program çalışır. Uyarıları okumak bu haftanın en önemli alışkanlığıdır.

Tablo 5.12
Hata Avcısı: hata, ileti (kısaltılmış; tam metinler kodlar/hata_avcisi/ciktilar/), sonuç ve çözüm

Hata	Derleyici iletisi (GCC 13)	Ne oldu? (gerçek çalışma)	Çözüm
HA1 `if (kalan_hak = 3)`	`suggest parentheses around assignment used as … [-Wparentheses]`	Atama yapıldı (3), koşul hep doğru: "Tam hakkınız var: 3"	`==` kullanın
HA2 `if (puan >= 60);`	`suggest braces around empty body in … [-Wempty-body]` + `this 'if' clause does not guard... [-Wmisleading-indentation]`	`;` boş gövdedir; blok hep çalıştı (puan 40): "Tebrikler, geçtiniz!"	`;`'yi silin
HA3 parantezsiz çok satırlı gövde	`this 'if' clause does not guard... [-Wmisleading-indentation]`	İkinci satır if'e ait değil: "Uyarı sayısı: 1" (final 70!)	Her zaman `{ }`
HA4 goto fail (yeniden kurgu)	`this 'if' clause does not guard... [-Wmisleading-indentation]`	2. adım (imza doğrulama) hiç çalışmadı; sonuç BAŞARILI	`{ }` + olumsuz test
HA5 sallanan else	`suggest explicit braces to avoid ambiguous … [-Wdangling-else]`	else iç if'e bağlandı; yas 16 için uyarı yazılmadı	Dış ve iç if'e `{ }`
HA6 `0 < x < 10`	`comparison of constant '10' with boolean … [-Wbool-compare]` + `comparisons like 'X<=Y<=Z' do not have … [-Wparentheses]`	`(0 < x)` 0/1 üretir, o da hep < 10: "25, (0, 10) aralığında." (x = 25)	`0 < x && x < 10`
HA7 `break` unutma	`this statement may fall through [-Wimplicit-fallthrough=]` (yalnızca `-Wextra` ile)	Seçim 1 iken "Not SİLME" de açıldı	Her case'i `break` ile bitirin
HA8 `toplam == 0.3`	Uyarı YOK; yalnızca `-Wfloat-equal` ile	"Eşit değil! toplam - 0.3 = 5.551e-17"	`fabs(a - b) < TOLERANS`
HA9 blok dışı erişim	`'fark' undeclared (first use in this …`	Derleme başarısız	Değişkeni bloktan önce bildirin

5.1 Yakından: parantezsiz gövde ve goto fail

HA3'te `if (final_notu < 50)` satırının altındaki iki satır aynı girintidedir. Girinti, ikinci satırın koşula bağlı olduğunu "söylüyor"; derleyici için ise `if` yalnızca bir sonraki **deyimi** korur. GCC'nin `-Wmisleading-indentation` uyarısı (`-Wall` içinde) tam olarak bu tutarsızlığı arar: "this 'if' clause does not guard… …this statement, but the latter is misleadingly indented as if it were guarded by the 'if'". Büyük Sorudaki açığı yeniden kuran HA4'te (aşağıda; akış şeması sunumda) aynı uyarı çıkar ve program "Sonuç: doğrulama BAŞARILI, bağlantı güvenli sayıldı" yazar: 2. adım hiç çalışmamıştır.

hata_avcisi/H05_HA4_goto_fail_yeniden.c · satır 14–20

```
14      if ((hata = ozet_hatasi) != 0)
15          goto fail;
16          goto fail;             /* yinelenen satır: HER durumda atlar */
17
18      printf("2. adım: sunucunun imzası doğrulanıyor\n");
19      if (imza_gecerli == 0) {
20          hata = -1;
```

Düzeltilmiş sürümde (`H05_HA4_goto_fail_yeniden_duzeltilmis.c`) `if` gövdesi süslü parantez içindedir; aynı kopyalama hatası (yinelenen `goto fail;`) **blok içinde** kaldığı için zararsızdır ve 2. adım çalışır: "Sonuç: doğrulama BAŞARISIZ, bağlantı reddedildi". `goto` deyimini bu derste kullanmıyoruz (Dijkstra, 1968); burada yalnızca olayı sadık biçimde yeniden kurmak için yer alıyor.

5.2 Uyarı vermeyen hata: kayan noktalı eşitlik

Uyarı vermeyen hata (HA8). 0.1 + 0.2 tam olarak 0.3 değildir (H3); `if (toplam == 0.3)` yanlış çıkar: "Eşit değil! toplam - 0.3 = 5.551e-17". `-Wall -Wextra` **uyarı vermez;** ayrı `-Wfloat-equal` seçeneği "comparing floating-point with '==' or '!=' is unsafe" der. Çözüm, farkın mutlak değerini bir **toleransla** karşılaştırmaktır (`fabs(toplam - 0.3) < TOLERANS`, düzeltilmiş dosya); çok büyük/küçük sayılarda göreli tolerans gerekir.

6. Mühendis Gözüyle

6.1 goto fail'den dört mühendislik dersi

- **Süslü parantez her zaman:** yinelenen ya da sonradan eklenen satır blokta kalır (HA4 düzeltilmiş). **Uyarıları hata say:** `-Werror` uyarılı kodun hiç üretilmemesini sağlar; sürekli tümleştirme (CI) hatlarının standart adımıdır.
- **Olumsuz testi yaz:** "geçerli imza kabul ediliyor mu?" testi goto fail'i yakalamaz; "sahte imza **reddediliyor** mu?" testi hemen yakalardı. **Erişilemez kod** bir uyarı işaretidir; kod incelemesi ve statik çözümleme araçları bunu arar.

6.2 Güvenlik: girdiye asla güvenme

Girdi doğrulama **saldırılara** karşı da ilk savunma hattıdır: geçersiz bir puan yanlış rapor üretir, geçersiz bir dizi indeksi (H10) ya da uzunluk (H13) belleği bozabilir. Dışarıdan gelen her değer kullanılmadan önce denetlenir; güvenlik kararlarında varsayılan yanıt "hayır"dır (Örnek 11'de yalnızca tanınan iki durumda erişim verilir; Carnegie Mellon University Software Engineering Institute, 2016).

6.3 Okunabilirlik ve doğruluk

- **Erken dönüş** (geçersiz durumları başta `return 1` ile elemek) derin iç içe if'lerden okunaklıdır. **Sihirli sayı yok:** eşikler adlandırılmış sabittir (`KADEME1_UST`, `FINAL_BARAJI`).
- **Para ve kesirli sınırlar:** Kayan noktalı değerler sınırda şaşırtabilir (HA8); para hesaplarında kuruş cinsinden tamsayı, not ortalamasında açık bir yuvarlama kuralı (`(int)(ortalama + 0.5)`) kullanılır. **Taşınabilirlik:** MinGW-w64 GCC aynı uyarıları verir; assembly işlemciye bağlıdır (ARM'da `cmp` + `b.le`), kalıp aynıdır.

7. Mini-OBS Projesi: v0.5

Mini-OBS ilk kez **karar verir**: notları doğrular, harf notunu ve durumu kurum tablosuna göre hesaplar.

> **MİNİ-OBS PROJESİ · v0.5 gereksinimleri**
>
> - v0.4 kuralları: `VIZE_AGIRLIK = 0.40`, `FINAL_AGIRLIK = 0.60`, `GECME_ORTALAMA = 60`, `FINAL_BARAJI = 50` sabitleri (yeni: `SARTLI_ORTALAMA = 50`); `ortalama = VIZE_AGIRLIK * vize + FINAL_AGIRLIK * final`; `yuvarlanmis = (int)(ortalama + 0.5)`.
> - **Girdi doğrulama:** vize ve final 0–100 dışındaysa açıklayıcı bir hata iletisi yazdırıp `return 1;` ile bitirin (tekrar sorma H6'da döngüyle gelecek).
> - **Harf notu:** dokuz kademe (Tablo 5.1), `yuvarlanmis` üzerinden else-if merdiveniyle (yuvarlama: ders içi modelleme varsayımı).
> - **Final barajı ve durum:** `final < FINAL_BARAJI` ise harf notu ne olursa olsun FF ve durum "BAŞARISIZ (final barajı)"; aksi hâlde `yuvarlanmis >= 60` ise "BAŞARILI", `yuvarlanmis >= 50` ise "ŞARTLI BAŞARILI (YANO ≥ 2,00 ise)", değilse "BAŞARISIZ".
> - Raporda: ortalama (`%6.2f`), yuvarlanmış ortalama, harf notu, durum; etiketler v0.4 ile aynı (`Vize (%40) :`, `Final (%60) :`); başlık "MİNİ-OBS v0.5 · Öğrenci Not Sistemi".

İpuçları. (1) Vize okunur okunmaz doğrulansın; geçersizse final sorulmasın. (2) Final barajını merdivenin **ilk basamağı** yapmak "ne olursa olsun FF" kuralını doğal olarak uygular. (3) 59.50 ile 59.40 ortalamaları ve 49 ile 50 final notları neden ayrı test edilmeli?

Tablo 5.13
Mini-OBS v0.5 kabul testleri (referans çözümle gerçek çalıştırma)

Test	Senaryo	vize, final	Beklenen	Referans çözümün gerçek çıktısı	
T1	AA (üst aralık)	92, 95	AA · BAŞARILI	93.80 → 94 · AA · BAŞARILI	✓
T2	sınır: 59.50 → 60 CC	70, 52.5	CC · BAŞARILI	59.50 → 60 · CC · BAŞARILI	✓
T3	sınır: 59.40 → 59 DC şartlı	69, 53	DC · ŞARTLI BAŞARILI	59.40 → 59 · DC · ŞARTLI BAŞARILI (YANO ≥ 2,00 ise)	✓
T4	final barajı (final 45)	100, 45	FF · BAŞARISIZ (final barajı)	67.00 → 67 · FF · BAŞARISIZ (final barajı)	✓
T5	geçersiz vize (101)	101	hata iletisi, kod 1	HATA (vize), çıkış kodu 1	✓
T6	geçersiz final (-5)	80, -5	hata iletisi, kod 1	HATA (final), çıkış kodu 1	✓
T7	sınır: final tam 50 (baraj)	80, 50	CC · BAŞARILI	62.00 → 62 · CC · BAŞARILI	✓
T8	sınır: final 49 (baraj altı)	80, 49	FF · BAŞARISIZ (final barajı)	61.40 → 61 · FF · BAŞARISIZ (final barajı)	✓
T9	sınır: 0 ve 100 geçerli	0, 100	CC · BAŞARILI	60.00 → 60 · CC · BAŞARILI	✓
T10	düşük ortalama (DD)	30, 60	DD · BAŞARISIZ	48.00 → 48 · DD · BAŞARISIZ	✓
T11	sınır: 80.60 → 81 BA	80, 81	BA · BAŞARILI	80.60 → 81 · BA · BAŞARILI	✓

Kabul ölçütleri: sıfır uyarı; T1–T11 geçer; geçersiz notta çıkış kodu 1; rapor v0.4 hizasında. Referans çözüm ve uçbirim dökümleri `91_Proje_MiniOBS/v0.5/` altındadır. **Bilinçli sınır:** harf girilirse (`abc`) `scanf` başarısız olur, not 0.0 kalır; dönüş değeri denetimi v0.6'da (H6).

8. Alıştırmalar ve Atölye

8.1 Atölye etkinlikleri

1. **Üçlü görev (eşli):** (a) Örnek 6'yı `double` puanla, (b) Örnek 7'yi "dar/geniş açılı" sınıflandırmasıyla, (c) Örnek 9'u `%` işlemiyle genişletin; her değişiklikten sonra test tablonuzu güncelleyin.
2. **Önce test tablosu:** Örnek 12'ye dördüncü kademe (500 kWh üstü 6,00 TL) eklenecek. **Kodu yazmadan önce** her dal için bir, her sınır için üç test hazırlayıp beklenenleri elle hesaplayın; sonra kodu yazıp çalıştırın.
3. **"Hangi dal çalışır?" kartları:** Web sayfasındaki kartları önce bireysel, sonra eşli yanıtlayın; derleyicinin uyarı verip vermeyeceğini de tahmin edin (kodlar: `kodlar/quiz/`, `kodlar/hata_avcisi/`).

8.2 Alıştırmalar

Alıştırma 5.1 ★☆☆ *Çıktı tahmini*

`x = 7`, `y = 12` iken aşağıdaki kod ne yazdırır? `x = 6` olsaydı? Hangi `x` değerleri için "E" yazılır?

alistirmalar/H05_A01_cikti_tahmini_ic_ice.c · satır 11–23

```
11      if (x > y) {
12          printf("A\n");
13      } else if (x + 5 >= y) {
14          printf("B\n");
15          if (x % 2 == 0) {
16              printf("C\n");
17          } else {
18              printf("D\n");
19          }
20      } else {
21          printf("E\n");
22      }
23      printf("F\n");
```

Alıştırma 5.2 ★★☆ *Çıktı tahmini*

`switch (kod)` gövdesi aşağıdadır. `kod = 2` için `puan` kaç olur? `kod` 1, 4 ve 7 olsaydı? Derleyici uyarı verir mi, neden?

alistirmalar/H05_A02_cikti_tahmini_switch.c · satır 12–25

```
12      case 1:
13          puan = puan + 10;
14          /* fall through */
15      case 2:
16          puan = puan + 20;
17          /* fall through */
18      case 3:
19          puan = puan + 30;
20          break;
21      case 4:
22          puan = puan + 40;
23          break;
24      default:
25          puan = -1;
```

Alıştırma 5.3 ★★☆ *Hata bul*

Program 95 puan için "şartlı başarılı" yazıyor. Neden? Hangi dal hiç çalıştırılamaz (dal kapsamı)? Düzeltin.

alistirmalar/H05_A03_yanlis_sira.c · satır 14–20

```
14      if (puan >= 50) {
15          printf("şartlı başarılı\n");
16      } else if (puan >= 60) {
17          printf("başarılı\n");
18      } else {
19          printf("başarısız\n");
20      }
```

Alıştırma 5.4 ★★☆ *Hata bul + test tasarla*

Yarıyıl sonu (final) sınavından en az 50 almak zorunludur (RTEÜ, 2025). Program 70 için doğru çalışıyor. 0–100 aralığı için bir sınır değer test tablosu hazırlayın (beklenenleri kuraldan elle yazın); hangi test hatayı yakalar?

alistirmalar/H05_A04_sinir_hatasi.c · satır 16–20

```
16      if (final_notu > 50) {
17          printf("baraj geçildi\n");
18      } else {
19          printf("FF (final barajı)\n");
20      }
```

Alıştırma 5.5 ★★☆ *Parsons*

Satırları, sıcaklığı “Dondurucu” (< 0), “Soğuk” (< 15), “Ilık” (< 25) ve “Sıcak” olarak sınıflandıran bir programa dönüştürün. Bir satır fazladır (çeldirici). Dosya bu hâliyle derlenmez.

alistirmalar/H05_A05_parsons_sicaklik.c · satır 9–22

```
 9      /* Satırları doğru sıraya dizin; biri fazladır (çeldirici). */
10      } else if (derece < 25.0) {
11          printf("Soğuk\n");
12      if (derece < 0.0) {
13      scanf("%lf", &derece);
14          printf("Sıcak\n");
15      } else if (derece < 15.0) {
16      } else if (derece > 25.0) {
17      double derece = 0.0;
18          printf("Ilık\n");
19      } else {
20          printf("Dondurucu\n");
21      printf("Sıcaklık (°C): ");
22      }
```

Alıştırma 5.6 ★★☆ *Kod tamamla*

Boşlukları doldurun: `mesaide` 9–17 arası (9 dahil, 17 hariç) için 1 olsun; “mesai dışı” koşulunu `!` kullanmadan De Morgan ile yazın. Dosya bu hâliyle derlenmez.

alistirmalar/H05_A06_tamamla_de_morgan.c · satır 14–19

```
14      int mesaide = (saat >= 9 ____ saat < 17);
15
16      /* Mesai DIŞI koşulu: ! kullanmadan, De Morgan ile yazın. */
17      if (saat ____ 9 ____ saat ____ 17) {
18          printf("Mesai dışı: kapıyı kartınızla açın.\n");
19      }
```

Alıştırma 5.7 ★★☆ *Kod yaz*

H3 Örnek 10’daki VKİ programını genişletin: VKİ < 18,5 “zayıf”, 18,5 ≤ VKİ < 25 “normal”, 25 ≤ VKİ < 30 “fazla kilolu”, ≥ 30 “obez” (Dünya Sağlık Örgütü’nün yetişkin sınıfları; World Health Organization [WHO], 2000). Kilo ve boyu doğrulayın (pozitif olmalı). Her sınıf ve her sınır için test tablosu hazırlayın.

Alıştırma 5.8 ★★☆ *Kod yaz*

Ay numarasını okuyup mevsimi yazan bir program yazın (Aralık–Şubat kış, Mart–Mayıs ilkbahar, Haziran–Ağustos yaz, Eylül–Kasım sonbahar). `switch` ve art arda case etiketleri kullanın; geçersiz ayı `default` ile reddedin.

Alıştırma 5.9 ★★★ *Hata bul + sınır testi*

Bu program Örnek 12'deki tarifeyi uygulamaya çalışıyor. 150 ve 151 kWh için çıktıları hesaplayın; sorun nedir? Örnek 12'nin test tablosunu bu programa uygulayın ve düzeltin.

alistirmalar/H05_A07_kademe_hatasi.c · satır 12–19

```
12      double bedel = 0.0;
13      if (tuketim <= 150) {
14          bedel = tuketim * 2.00;
15      } else if (tuketim <= 300) {
16          bedel = tuketim * 3.00;
17      } else {
18          bedel = tuketim * 4.50;
19      }
```

Alıştırma 5.10 ★★★ *Kod yaz*

Gün, ay ve yıl okuyup tarihin geçerli olup olmadığını bulan bir program yazın (Örnek 5 ve Örnek 10'u birleştirin). En az 10 testlik bir tablo hazırlayın: 29.02.2024 (geçerli), 29.02.2023, 31.04.2026, 0.1.2026, 31.12.2026, 1.13.2026, 29.02.1900, 29.02.2000 …

9. Kendini Değerlendirme

- ☐ C'de hangi değerlerin doğru/yanlış sayıldığını açıklayabiliyor; if, if-else, else-if, iç içe if, switch ve ?: arasından uygun yapıyı gerekçesiyle seçebiliyorum.
- ☐ Sallanan else'i ve parantezsiz gövde tuzağını tanıyor, GCC uyarılarını okuyup düzeltebiliyor; switch'te break, default ve bilinçli düşüşü doğru kullanabiliyorum.
- ☐ Bileşik koşulları De Morgan ile dönüştürebiliyor; her dalı ve her sınırı (bir altı, kendisi, bir üstü) kapsayan test tablosu hazırlayabiliyorum.
- ☐ `gcc -S` çıktısında if'in `cmp` + koşullu atlamaya, yoğun switch'in atlama tablosuna dönüştüğünü gösterebiliyorum.

10. AP-2 Köprüsü ve Merak Kutusu

AP-2 KÖPRÜSÜ · Karşılaştırmadan sıralamaya

- **Karşılaştırma**, AP-2'deki karşılaştırmaya dayalı sıralama algoritmalarının temel işlemidir; algoritmanın maliyeti çoğu zaman karşılaştırma sayısıyla ölçülür (Örnek 4'te saydık).
- **Karar ağaçları** (Şekil 5.3) AP-2'de "karşılaştırmaya dayalı sıralama en az kaç karşılaştırma gerektirir?" sorusunun görsel zeminidir: her karşılaştırma bir düğüm, her olası sıralama bir yapraktır. Seyrek switch'te GCC'nin değerleri "ortadan bölerek" araması ikili arama sezgisine benzer.

MERAK KUTUSU · **Merak kutusu**

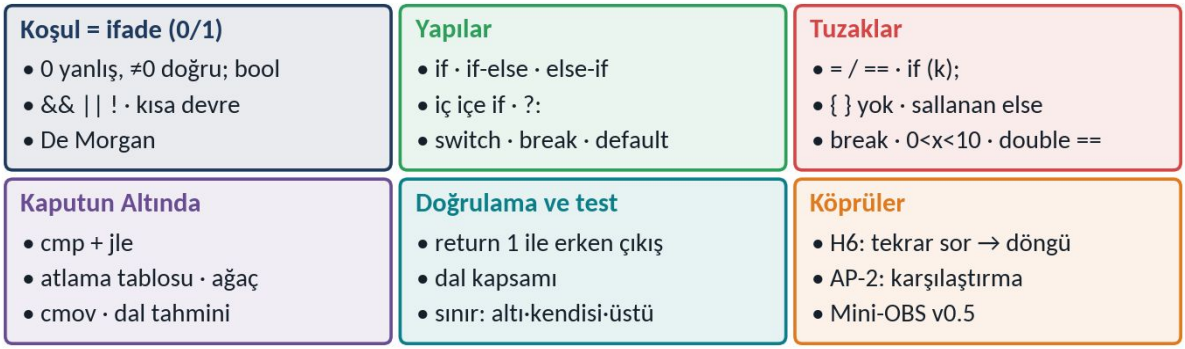
- **cmov:** koşullu taşımanın ne zaman daha hızlı olduğu verinin öngörülebilirliğine bağlıdır (Patterson & Hennessy, 2017; Bryant & O'Hallaron, 2016). **goto:** Dijkstra (1968) `goto`'nun sınırsız kullanımını eleştirdi; C'de bugün en yaygın meşru kullanımı hata yollarında kaynak serbest bırakmaktır. goto fail hatası `goto`'nun değil, parantezsiz `if`'in ve yinelenen satırın sonucudur.

11. Özet, Kavram Haritası ve Terim Sözlüğü

11.1 Haftanın özeti

- C'de koşul bir ifadedir: 0 yanlış, sıfır dışı doğru; ilişkisel ve mantıksal operatörler int 0/1 üretir; `stdbool.h` `bool`, `true`, `false` sağlar.
- `if`, `if-else`, `else-if` merdiveni (aralıklar büyükten küçüğe), iç içe `if` (`else` en yakın `if`'e aittir), `switch` (case = giriş noktası, `break` = çıkış) ve `?:`; gövdeler her zaman `{ }` içinde.
- De Morgan olumsuz koşulları sadeleştirir; kısa devre sıfıra bölmeyi önler. Girdi kullanılmadan önce doğrulanır; testler her dalı ve her sınırı (altı, kendisi, üstü) kapsar.
- Makinede if = `cmp` + koşullu atlama; yoğun switch = atlama tablosu; seyrek switch = karşılaştırma ağacı; basit seçim = `cmov`.

Şekil 5.9
Hafta 5 kavram haritası (ayrıntılı ilişki diyagramı sunumda ve web sayfasında)



11.2 Büyük Sorunun yanıtı

Fazladan `goto fail;` satırı, parantezsiz bir `if`'ten sonra geldiği için koşulsuz bir atlamaya dönüştü ve imza doğrulamasını erişilemez kıldı; hata değişkeni 0 olduğundan doğrulama "başarılı" sayıldı. Önleyecek araçların hepsi bu haftanın konusudur: süslü parantez, derleyici uyarıları (`-Wmisleading-indentation`), sahte imzanın **reddedildiğini** sınayan olumsuz testler ve kod incelemesi (Langley, 2014; NIST, 2014).

11.3 Sonraki hafta: Döngüler I

Geçersiz puanda yapabildiğimiz tek şey programı sonlandırmaktı; "geçersizse tekrar sor" ancak okuma ve doğrulamayı **tekrarlayan** bir yapıyla, döngüyle mümkündür. H6'da `while`, `do-while` ve `for`'u bu haftanın koşullarıyla öğreneceğiz; Mini-OBS v0.6 N öğrenciyi işleyecek.

11.4 Terim sözlüğü (Bölüm 6.8 terim standardıyla uyumlu)

Tablo 5.14
Haftanın terimleri

Türkçe	İngilizce	Kısa açıklama
karar yapısı; koşul	selection statement; condition	if / switch; 0 ya da sıfır dışı değer üreten ifade
blok; kapsam	block; scope	`{ }` içindeki deyimler; bir adın görünür olduğu bölge
else-if merdiveni; iç içe if; sallanan else	else-if ladder; nested if; dangling else	sırayla sınanan koşullar; else en yakın if'e aittir
koşul operatörü; düşüş	conditional operator; fall-through	`k ? a : b`; break olmadan alttaki case'e geçme
kısa devre değerlendirme	short-circuit evaluation	&& / || sağ tarafı gerektiğinde hesaplanır
De Morgan kuralları	De Morgan's laws	!(A && B) ≡ !A || !B
girdi doğrulama; savunmacı programlama	input validation; defensive programming	girdiyi kullanmadan önce denetleme
dal kapsamı; sınır değer çözümlemesi	branch coverage; boundary value analysis	her dal için test; sınırda üç test
koşullu atlama; atlama tablosu; dal tahmini	conditional jump; jump table; branch prediction	`jle`; switch için adres tablosu; atlamanın önceden tahmini

12. Kaynaklar

Bryant, R. E., & O'Hallaron, D. R. (2016). *Computer systems: A programmer's perspective* (3rd ed.). Pearson.

Carnegie Mellon University Software Engineering Institute. (2016). *SEI CERT C coding standard: Rules for developing safe, reliable, and secure systems* (2016 ed.). Carnegie Mellon University.

Dijkstra, E. W. (1968). Go to statement considered harmful. *Communications of the ACM*, *11*(3), 147–148. https://doi.org/10.1145/362929.362947

ISO/IEC. (2018). *Information technology — Programming languages — C* (ISO/IEC Standard No. 9899:2018). International Organization for Standardization.

Langley, A. (2014, February 22). *Apple's SSL/TLS bug*. ImperialViolet. https://www.imperialviolet.org/2014/02/22/applebug.html

National Institute of Standards and Technology. (2014). *CVE-2014-1266 detail*. National Vulnerability Database. https://nvd.nist.gov/vuln/detail/CVE-2014-1266

Patterson, D. A., & Hennessy, J. L. (2017). *Computer organization and design RISC-V edition: The hardware/software interface*. Morgan Kaufmann.

Recep Tayyip Erdoğan Üniversitesi. (2025). *Recep Tayyip Erdoğan Üniversitesi ön lisans ve lisans eğitim-öğretim ve sınav yönetmeliği* (Resmî Gazete, 14 Aralık 2025, Sayı: 33107; son değişiklik: Resmî Gazete, 27 Şubat 2026, Sayı: 33181).

World Health Organization. (2000). *Obesity: Preventing and managing the global epidemic* (WHO Technical Report Series No. 894). World Health Organization.